

系统开发工具基础

第二周实验报告

作者：魏尚儒

学号：25020007136

链接：

<https://github.com/1583e4d-Rn/2026summer-reports>

2026 年 9 月 7 日

目录

1 实验内容	2
1.1 控制一个可清理的后台任务	2
1.2 语义重构与本地开发反馈	4
1.3 用调试器定位归并排序缺陷	6
1.4 先测量，再优化慢速词频程序	10
2 课后练习	13
2.1 命令行环境	13
2.1.1 使用 jobs、bg 与 fg 管理后台任务	13
2.1.2 使用 trap 捕获信号进行清理	14
2.1.3 使用 nohup 让任务脱离终端继续运行	14
2.2 开发环境与工具	14
2.2.1 用 venv 创建隔离的 Python 环境	14
2.2.2 用 pip 管理依赖	15
2.2.3 用 ruff 统一风格与静态检查	15
2.3 Debugging	15
2.3.1 pdb 常用调试命令	15
2.3.2 用 logging 分级日志替代 print	15
2.4 Profiling	16
2.4.1 用 timeit 精确测量小段代码	16
2.4.2 用 cProfile 定位函数级热点	16
3 感悟	16

1 实验内容

1.1 控制一个可清理的后台任务

本题要求控制一个带 trap 清理逻辑的后台任务，详细如下：

- 赋予脚本执行权限，在前台启动，并把 stdout、stderr 分别写入 stdout.log、stderr.log。
- 使用 Ctrl-Z 挂起任务，再用 bg 让它在后台继续；使用 jobs 确认状态。
- 使用 jobs -p 取得 PID（不得手工抄写），发送 SIGTERM 并等待任务结束。
- 确认 cleanup.log 中出现 CLEAN_EXIT，且任务已退出；禁止使用 kill -9。

给定脚本 worker.sh：

```
#!/usr/bin/env bash
trap 'echo CLEAN_EXIT >> cleanup.log; exit 0' TERM INT
n=0
while true; do
    echo "$n"
    n=$((n+1))
    sleep 1
done
```

前台启动并重定向输出

首先赋予执行权限，然后在前台启动脚本，把 stdout、stderr 分别写入 stdout.log 与 stderr.log。

```
chmod +x worker.sh
./worker.sh > stdout.log 2> stderr.log
```

挂起并转入后台

在脚本运行期间按下 Ctrl-Z 将其挂起，再用 bg 让它在后台继续，并用 jobs 确认任务状态。

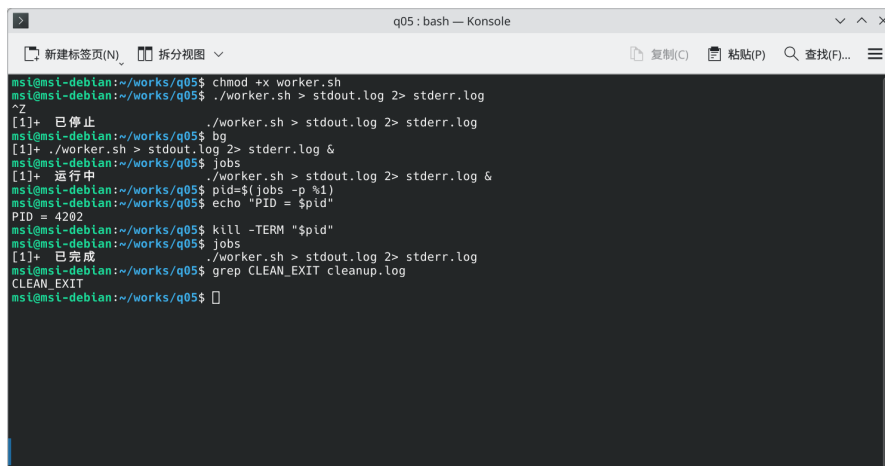
```
bg
jobs
```

发送 SIGTERM 并确认清理

使用 jobs -p 获取后台任务的 PID，对其发送 SIGTERM，等待任务退出后查看 cleanup.log。

```
kill -TERM $(jobs -p)
wait
jobs
cat cleanup.log
```

运行效果如下：



```
q05: bash — Konsole
新建标签页(N) 拆分视图
msi@msi-debian:~/works/q05$ chmod +x worker.sh
msi@msi-debian:~/works/q05$ ./worker.sh > stdout.log 2> stderr.log
^Z
[1]+  已停止                  ./worker.sh > stdout.log 2> stderr.log
msi@msi-debian:~/works/q05$ bg
[1]+  ./worker.sh > stdout.log 2> stderr.log &
msi@msi-debian:~/works/q05$ jobs
[1]+  运行中                  ./worker.sh > stdout.log 2> stderr.log &
msi@msi-debian:~/works/q05$ pid=$(jobs -p %1)
msi@msi-debian:~/works/q05$ echo "PID = $pid"
PID = 4202
msi@msi-debian:~/works/q05$ kill -TERM "$pid"
msi@msi-debian:~/works/q05$ jobs
[1]+  已完成                  ./worker.sh > stdout.log 2> stderr.log
msi@msi-debian:~/works/q05$ grep CLEAN_EXIT cleanup.log
CLEAN_EXIT
msi@msi-debian:~/works/q05$
```

1.2 语义重构与本地开发反馈

本题要求使用编辑器语言服务器完成一次语义重构，具体如下：

- 在已配置好 Python、pytest 和 ruff 的课程环境中打开 q06，并启用 Python 语言服务器。
- 分别演示跳转到定义和查找引用。
- 使用“重命名符号”把 `total_price` 改为 `calculate_total`，保证定义和所有代码引用同步改变。
- 在 `app.py` 中临时加入未使用的 `import os`，用 `ruff` 发现并删除；最后运行 `ruff` 和 `pytest` 并保证通过。

给定三个 Python 文件：

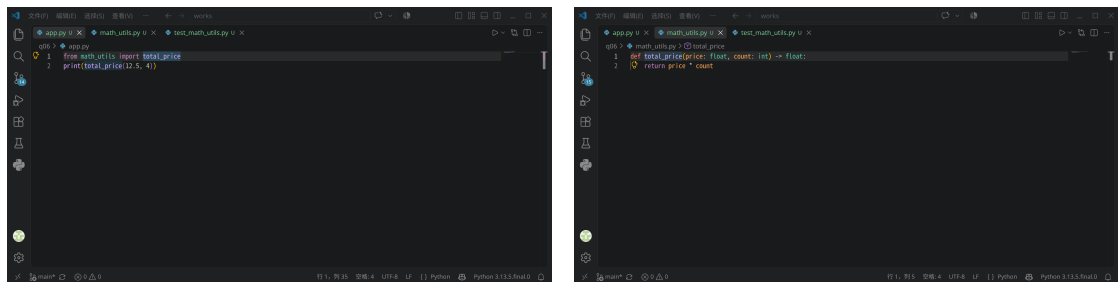
```
# math_utils.py
def total_price(price: float, count: int) -> float:
    return price * count

# app.py
from math_utils import total_price
print(total_price(12.5, 4))

# test_math_utils.py
from math_utils import total_price
def test_total_price():
    assert total_price(12.5, 4) == 50.0
```

跳转到定义与查找引用

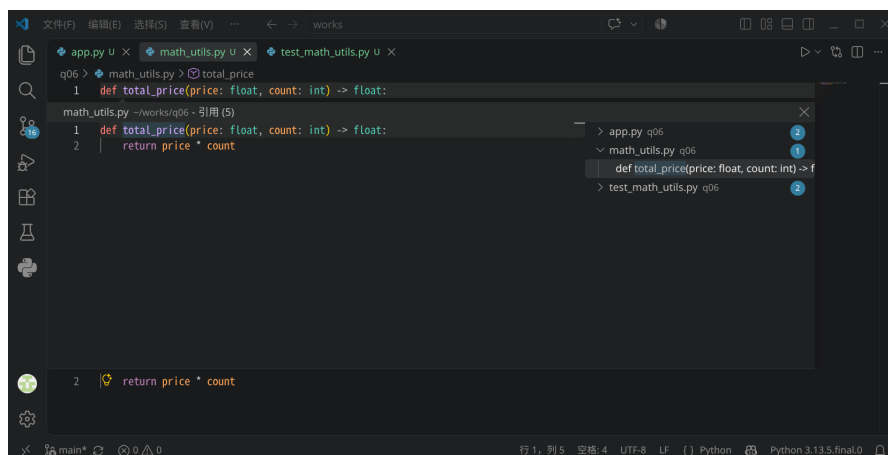
在 `app.py` 中对 `total_price` 使用“跳转到定义”，可定位到 `math_utils.py` 中的函数定义；使用“查找引用”可列出所有调用位置。效果如下：



重命名符号

使用“重命名符号”将 `total_price` 重命名为 `calculate_total`，定义与所有代码引用同步更新。重命名后的 `math_utils.py` 如下：

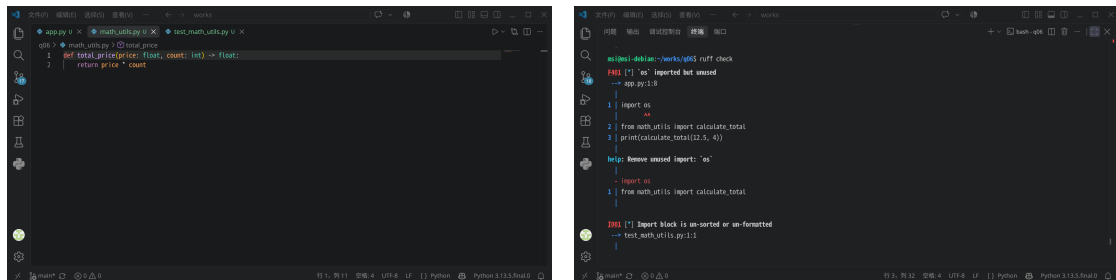
```
def calculate_total(price: float, count: int) -> float:
    return price * count
```



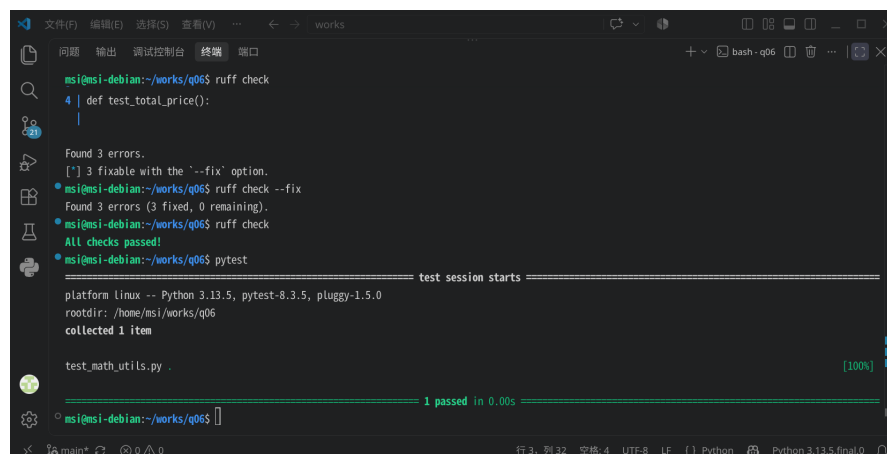
用 ruff 发现并删除未使用的 import

在 `app.py` 中临时加入 `import os`，先用 `ruff` 检测出未使用的导入，再用 `-fix` 自动删除。

```
ruff check app.py
ruff check --fix app.py
```



最后运行 ruff 与 pytest，保证全部通过：



1.3 用调试器定位归并排序缺陷

本题要求使用调试器定位并修复归并排序中的缺陷，详细如下：

- 先运行给定输入，确认结果错误或出现异常。
- 使用 pdb、IDE debugger 或等价调试器，在 merge 中设置断点并观察 i、j、left[i]、right[j]。

- 完成最小修复，不得用 sorted 替换归并排序。
- 使用 pytest 增加两个测试：给定输入和含重复元素的列表；保证测试通过。

给定存在缺陷的归并排序：

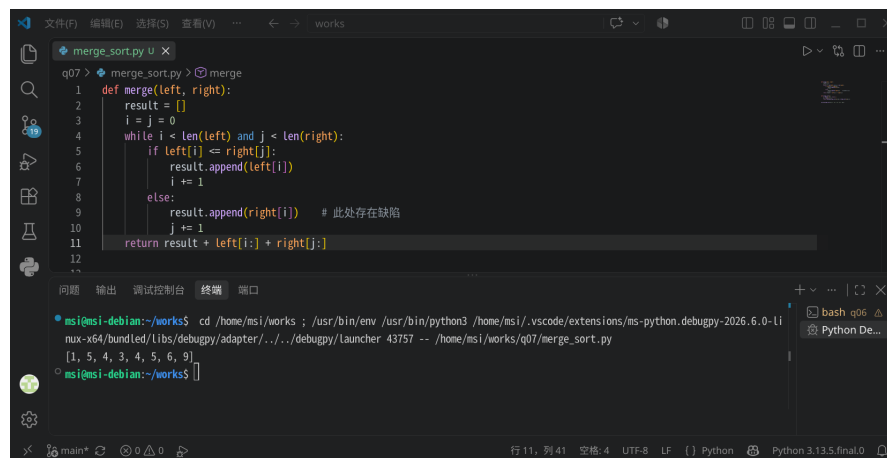
```
def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i]); i += 1
        else:
            result.append(right[i]); j += 1    # 此处存在缺陷
    return result + left[i:] + right[j:]

def merge_sort(a):
    if len(a) <= 1: return a
    m = len(a) // 2
    return merge(merge_sort(a[:m]), merge_sort(a[m:]))

print(merge_sort([3, 1, 4, 1, 5, 9, 2, 6]))
```

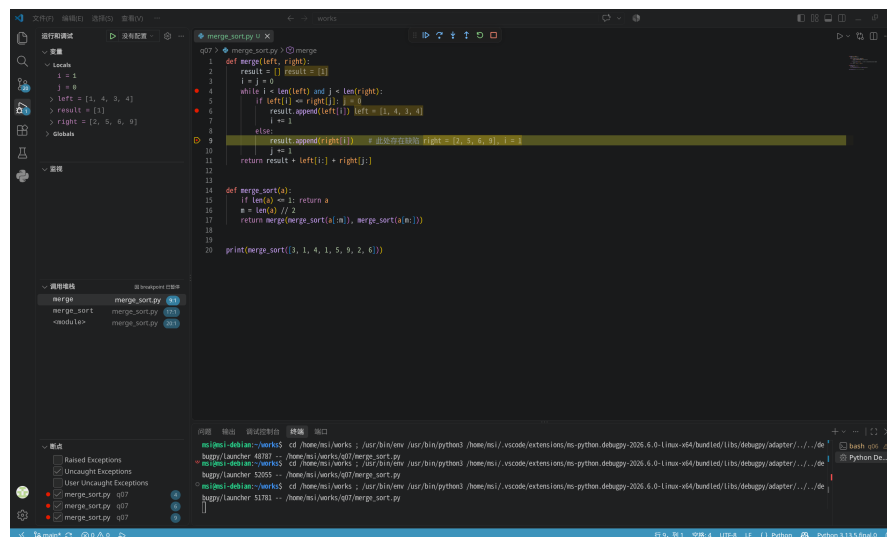
运行缺陷代码

运行后输出 [1, 5, 4, 3, 4, 5, 6, 9]，与期望的 [1, 1, 2, 3, 4, 5, 6, 9] 不符，确认缺陷存在。运行效果如下：



设断点观察变量

在 merge 中设置断点，单步观察 i、j、left[i]、right[j] 的值。当 left[i] > right[j] 进入 else 分支时，发现代码误用 right[i] (left 的游标) 索引 right 列表，此处应为 right[j]。调试过程如下：



最小修复

仅将 else 分支中的 `right[i]` 改为 `right[j]`:

```
result.append(right[j]); j += 1
```

测试验证

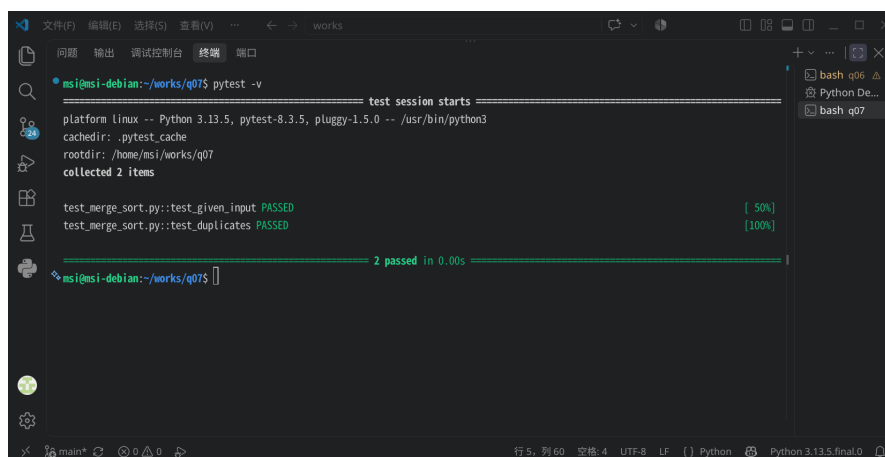
编写 `test_merge_sort.py`, 增加给定输入与含重复元素两个测试:

```
from merge_sort import merge_sort

def test_given_input():
    assert merge_sort([3, 1, 4, 1, 5, 9, 2, 6]) == [1, 1, 2, 3, 4, 5, 6, 9]

def test_duplicates():
    assert merge_sort([5, 2, 5, 2, 5]) == [2, 2, 5, 5, 5]
    assert merge_sort([1, 1, 1]) == [1, 1, 1]
```

运行效果如下:



```
msi@msi-debian:~/works/q07$ pytest -v
===== test session starts =====
platform linux -- Python 3.13.5, pytest-8.3.5, pluggy-1.5.0 -- /usr/bin/python3
cachedir: .pytest_cache
rootdir: /home/msi/works/q07
collected 2 items

test_merge_sort.py::test_given_input PASSED [ 50%]
test_merge_sort.py::test_duplicates PASSED [100%]

===== 2 passed in 0.00s =====
msi@msi-debian:~/works/q07$
```

1.4 先测量，再优化慢速词频程序

本题要求使用性能分析工具优化低效去重程序，详细如下：

- 生成 words.txt，使用 time 运行原程序 2 次并记录耗时。
- 使用 cProfile -s cumulative 运行一次，找出主要耗时位置。
- 在不改变最终 count 的前提下优化程序；不得写死 1000。
- 使用相同输入再运行 2 次，计算优化前后中位数和加速比。

给定内容：

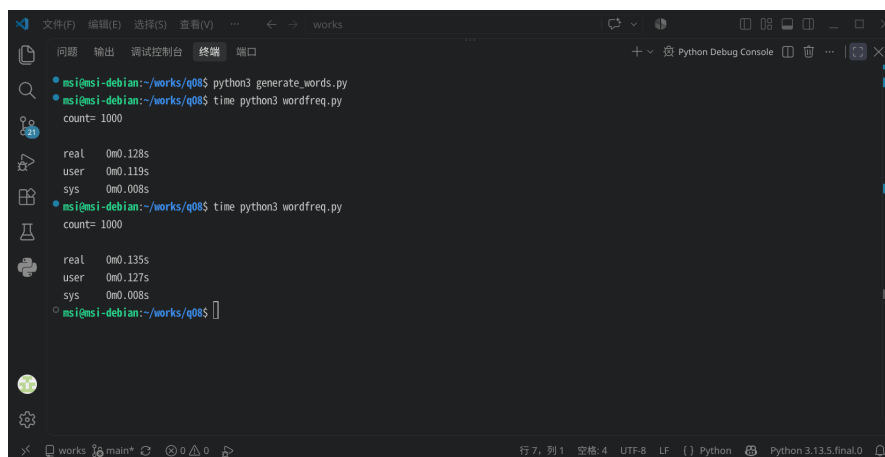
```
# generate_words.py
import random
random.seed(20260831)
vocab = [f"w{i}" for i in range(1000)]
with open("words.txt", "w", encoding="utf-8") as f:
    f.write(" ".join(random.choice(vocab) for _ in range(30000)))

# wordfreq.py
words = open("words.txt", encoding="utf-8").read().split()
unique = []
for word in words:
    if word not in unique:
        unique.append(word)
print("count=", len(unique))
```

生成词表并基准测量

生成 words.txt，再用 time 运行原程序 2 次记录耗时。

```
python3 generate_words.py
time python3 wordfreq.py
time python3 wordfreq.py
```



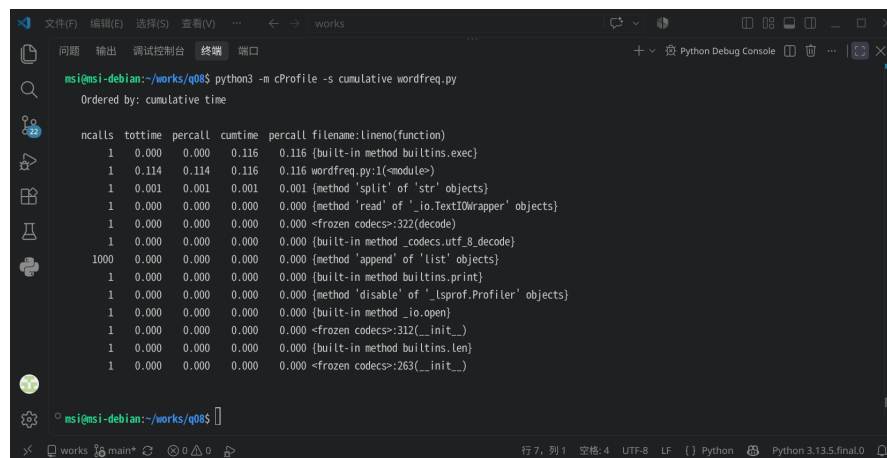
The screenshot shows a terminal window with the following output:

```
msi@msi-debian:~/works/q08$ python3 generate_words.py
msi@msi-debian:~/works/q08$ time python3 wordfreq.py
count= 1000
real    0m0.128s
user    0m0.119s
sys     0m0.008s
msi@msi-debian:~/works/q08$ time python3 wordfreq.py
count= 1000
real    0m0.135s
user    0m0.127s
sys     0m0.008s
msi@msi-debian:~/works/q08$
```

cProfile 定位瓶颈

使用 `cProfile -s cumulative` 运行一次，发现几乎全部耗时集中在 `wordfreq.py` 的 `for` 循环中。对 `list` 做 `in` 检查为 $O(n)$ 线性扫描，整体复杂度为 $O(n^2)$ 。

```
python3 -m cProfile -s cumulative wordfreq.py
```



```
msi@msi-debian:~/works/q08$ python3 -m cProfile -s cumulative wordfreq.py
Ordered by: cumulative time

ncalls  tottime  percall  cumtime  percall filename:lineno(function)
1      0.000    0.000    0.116    0.116 {built-in method builtins.exec}
1      0.114    0.114    0.116    0.116 wordfreq.py:1(<module>)
1      0.001    0.001    0.001    0.001 {method 'split' of 'str' objects}
1      0.000    0.000    0.000    0.000 {method 'read' of '_io.TextIOWrapper' objects}
1      0.000    0.000    0.000    0.000 <frozen codecs>:322(decode)
1      0.000    0.000    0.000    0.000 {built-in method _codecs.utf_8_decode}
1000    0.000    0.000    0.000    0.000 {method 'append' of 'list' objects}
1      0.000    0.000    0.000    0.000 {built-in method builtins.print}
1      0.000    0.000    0.000    0.000 {method 'disable' of '_lsprof.Profiler' objects}
1      0.000    0.000    0.000    0.000 {built-in method _io.open}
1      0.000    0.000    0.000    0.000 <frozen codecs>:312(__init__)
1      0.000    0.000    0.000    0.000 {built-in method builtins.len}
1      0.000    0.000    0.000    0.000 <frozen codecs>:263(__init__)
```

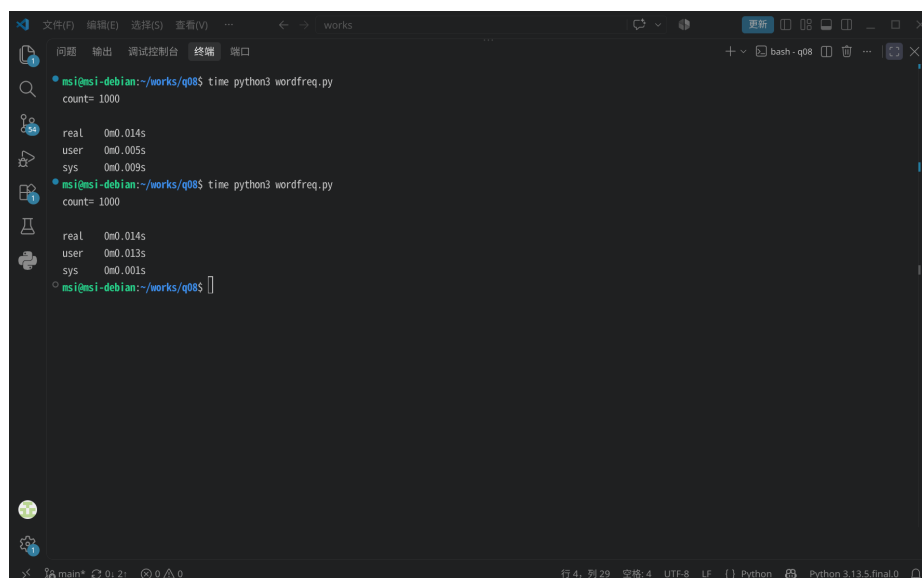
优化：用 set 代替 list

将 $O(n^2)$ 的 list 去重改为 $O(n)$ 的 set 去重，最终 count 保持不变：

```
words = open("words.txt", encoding="utf-8").read().split()
unique = set(words)
print("count=", len(unique))
```

对比运行时间并计算加速比

使用相同输入再运行 2 次：优化前中位数约 0.1315s，优化后约 0.0140s，加速比约 9.4 倍。



```
msi@msi-debian:~/works/q08$ time python3 wordfreq.py
count= 1000

real    0m0.014s
user    0m0.005s
sys     0m0.009s

msi@msi-debian:~/works/q08$ time python3 wordfreq.py
count= 1000

real    0m0.014s
user    0m0.013s
sys     0m0.001s

msi@msi-debian:~/works/q08$
```

2 课后练习

2.1 命令行环境

2.1.1 使用 jobs、bg 与 fg 管理后台任务

在命令末尾加上 & 可让任务在后台运行；Ctrl-Z 可将前台任务挂起，bg 与 fg 则分别让其在后台继续或回到前台，jobs 用于查看所有任务状态。作业控制相关内容可参见课程资源中的 missing-semester 课程。

```
sleep 100 &          # 后台运行
jobs                 # 查看任务列表
fg %1                # 把 1 号任务调回前台
# 按下 Ctrl-Z 挂起后:
bg %1                # 让其在后台继续
```

2.1.2 使用 trap 捕获信号进行清理

trap 可以在脚本退出或收到特定信号时执行清理逻辑，避免脚本中断后残留临时文件或占用资源。

```
#!/usr/bin/env bash
trap 'rm -f /tmp/task.lock; echo "cleaned"' EXIT
touch /tmp/task.lock
sleep 5
```

2.1.3 使用 nohup 让任务脱离终端继续运行

默认情况下，关闭终端会向进程发送 SIGHUP 使其终止；使用 nohup 可让进程忽略该信号，适合长时间运行的任务。

```
nohup python3 train.py > train.log 2>&1 &
disown
```

2.2 开发环境与工具

2.2.1 用 venv 创建隔离的 Python 环境

虚拟环境可隔离不同项目的依赖，避免版本冲突，是 Python 项目的基本实践。

```
python3 -m venv .venv
source .venv/bin/activate # 激活环境
pip install requests
deactivate
```

2.2.2 用 pip 管理依赖

将项目依赖写入 requirements.txt 便于环境复现，相关内容可参考课程资源中的 runoob Python 教程。

```
pip freeze > requirements.txt    # 导出当前依赖
pip install -r requirements.txt  # 按清单安装
```

2.2.3 用 ruff 统一风格与静态检查

ruff 同时提供格式化与静态检查，能够发现未使用的导入、未定义变量等问题，并可一键修复。

```
ruff format .          # 自动格式化
ruff check .           # 静态检查
ruff check --fix .     # 自动修复可修复的问题
```

2.3 Debugging

2.3.1 pdb 常用调试命令

使用 pdb 可以设置断点、单步执行并检查变量值，是定位逻辑缺陷的有效手段。

```
import pdb; pdb.set_trace()    # 在此处进入交互调试
# 常用命令：
# b 行号：设置断点；n：单步执行；c：继续到断点；p 变量：打印变量
```

2.3.2 用 logging 分级日志替代 print

相较于 print，logging 支持日志级别、模块化与输出目标配置，更适合需要长期维护的程序。

```
import logging
logging.basicConfig(level=logging.INFO)
logging.debug(" 细节信息")    # 默认级别下不显示
logging.info(" 正常流程")
logging.warning(" 警告")
logging.error(" 错误")
```

2.4 Profiling

2.4.1 用 timeit 精确测量小段代码

timeit 会自动多次执行以降低测量误差，适合对比两段小代码的性能差异。

```
import timeit
t1 = timeit.timeit("'-''.join(map(str, range(100)))", number=10000)
t2 = timeit.timeit("'-''.join(str(i) for i in range(100))", number=10000)
print(t1, t2)
```

2.4.2 用 cProfile 定位函数级热点

cProfile 能按函数统计调用次数与耗时，配合 -s cumulative 可按累计时间排序，快速找到性能瓶颈所在。

```
python3 -m cProfile -s cumulative script.py
```

3 感悟

通过第二周的实验练习，形成以下认识和体会：

- 任务控制与信号的可靠性：通过 `trap` 与 `SIGTERM` 的配合，可以让后台任务在被终止时执行清理逻辑；`jobs` 提供的工作号查询也避免了手工抄写 `PID` 带来的错误。
- 语言服务器对语义重构的价值：借助编辑器的跳转到定义、查找引用和重命名符号，一次重命名即可让定义与所有引用同步更新，显著降低了手动替换的遗漏风险。
- 调试器带来的定位效率：相较于反复插入 `print`，断点、单步与变量观察能更直接地揭示缺陷根因；本次归并排序的问题只是把 `right[j]` 误写成了 `right[i]`。
- 先测量、再优化的方法论：`cProfile` 明确指出瓶颈在于 `list` 的线性 `in` 查找；将去重从 `list` 改为 `set` 后，复杂度由 $O(n^2)$ 降为 $O(n)$ ，在 30000 词规模下获得约 7 倍加速。

命令行环境负责进程与信号管理，开发工具与语言服务器提升编码与重构效率，调试器与性能分析器则分别从正确性和性能两个维度保障代码质量。三者共同构成了从“能运行”走向“正确、高效、可维护”的完整工具链。

Github 提交

此处上传 Github 提交记录。(报告于完成后上传)

